



BÀI 06: QUẢN LÝ VÀ TRÌNH BÀY CÁC THÀNH PHẦN TRONG AWT VÀ SWING

- ❑ Layout Manager
 - ❑ Flow Layout
 - ❑ Border Layout
 - ❑ GridLayout
 - ❑ GridBag Layout
- ❑ Bài tập



LAYOUT MANAGER

- ❑ **Layout Manager** là một thành phần trong Java AWT và Swing được sử dụng để quản lý cách bố trí (layout) của các thành phần giao diện (components) trong một Container như **Frame**, **Panel**, hay **Applet**. Nó xác định cách sắp xếp, kích thước và vị trí của các thành phần bên trong container.
- ❑ Java cung cấp nhiều loại **Layout Manager** khác nhau để đáp ứng các yêu cầu bố trí giao diện người dùng một cách linh hoạt.
- ❑ Để thiết lập cho các Container theo một kiểu trình bày (Layout) ta sử dụng phương thức dùng **void setLayout(layout)** (Frame, JFrame, Panel, Applet..)



LAYOUT MANAGER

❑ Vai trò của Layout Manager

✓ Tự động quản lý kích thước và vị trí:

- Điều chỉnh vị trí và kích thước của các thành phần bên trong container khi cửa sổ được thay đổi kích thước.

✓ Đơn giản hóa việc lập trình giao diện:

- Giúp lập trình viên tránh phải tính toán thủ công vị trí và kích thước của từng thành phần.

✓ Hỗ trợ nhiều kiểu bố trí:

- Phù hợp với các thiết kế giao diện khác nhau.



❑ Flow Layout (Mặc định với Panel)

Kiểu Flow Layout xếp các đối tượng trên một hướng theo dòng. Nếu đối tượng mới thêm không đủ chỗ (chiều rộng) thì nó sẽ tự động thêm vào đầu dòng mới.

❑ Các phương thức tạo lập:

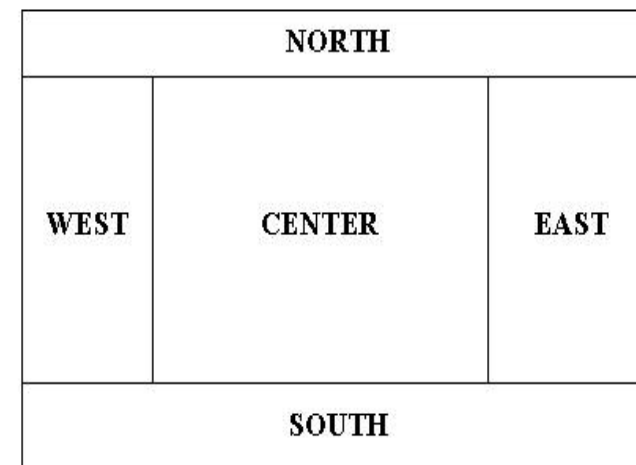
- **FlowLayout():** Khởi tạo đối tượng trình bày mặc định.
- **FlowLayout(int):** Khởi tạo đối tượng trình bày với cách căn lề xác định.
- **FlowLayout(int, int, int):** Khởi tạo với ba tham số: Thứ nhất là cách căn lề, thứ hai là khoảng cách giữa hai dòng (chiều cao), thứ ba là khoảng cách giữa hai đối tượng (chiều ngang).
- ✓ Tham số căn lề có thể nhận một trong ba giá trị:
 - **FlowLayout.LEFT:** Căn lề trái, là giá trị mặc định.
 - **FlowLayout.CENTER:** Căn lề giữa.
 - **FlowLayout.RIGHT:** Căn lề phải.



❑ Border Layout (Mặc định với Frame, Jframe)

Cách hiển thị Border Layout sẽ chia frame thành 5 vùng cố định và tự động kéo dãn các vùng sao cho chiếm hết bề mặt của frame:

- **West:** Vùng phía tây, tức là phía lề bên trái.
- **East:** Vùng phía đông, tức là phía lề bên phải.
- **North:** Vùng phía bắc, tức là phía lề trên.
- **South:** Vùng phía nam, tức là phía lề dưới.
- **Center:** Vùng trung tâm, ở chính giữa frame.





❑ Border Layout

Các phương thức cơ bản của lớp BorderLayout:

- **BorderLayout():** Khởi tạo một đối tượng trình bày theo cách BorderLayout.
- Khi một **Container** được trình bày theo cách BorderLayout, ta có thể dùng phương thức sau để gắn các đối tượng vào các vùng của Container

<Đối tượng Container>.add(<Đối tượng component>, <Vùng border>);

Ví dụ: `myFrame.add(new Button("Click"), "Center");`

sẽ gắn vào vùng trung tâm của myFrame một nút nhấn có tên là "Click".



❑ Border Layout

Lưu ý:

- Cách trình bày BorderLayout luôn chia container thành 5 vùng xác định.
- Nếu gắn nhiều đối tượng vào cùng một vùng, chỉ có đối tượng gắn sau là nhìn thấy được.
- **Nếu muốn trong một vùng chứa được nhiều đối tượng, ta có thể gắn vào mỗi vùng một Panel. Sau đó trong panel, ta chọn cách trình bày riêng cho panel và gắn các đối tượng vào panel.**



❑ GridLayout

- Đối với một container trình bày theo kiểu GridLayout thì: Bộ trình bày tạo một khung lưới vô hình với các ô bằng nhau.
- Các đối tượng sẽ đặt vừa kích thước với từng ô đó. Thứ tự sắp xếp từ trái qua phải, từ trên xuống dưới .

❑ GridLayout

- Phương thức cơ bản:
 - **GridLayout():** Khởi tạo mặc định. Số cột sẽ tự động tăng tương ứng với số phần tử được thêm vào
 - **GridLayout(dong int,cot int):** Khởi tạo một đối tượng trình bày. Hai tham số đầu vào lần lượt là số hàng và số cột của grid trình bày.
 - **GridLayout(int, int, int, int):** Khởi tạo một đối tượng trình bày, hai tham số đầu xác định số hàng và số cột trình bày. Hai tham số sau xác định khoảng cách giữa các dòng và các cột của bảng.

❑ GridLayout

- Khi số lượng đối tượng được chèn nhiều hơn vào frame, ta muốn chương trình tự tính số hàng, hoặc tự tính số cột hiển thị, thì ta để tham số tương ứng là 0. VD:
 - `setLayout(new GridLayout(3,0));` // cố định số hàng là 3, số cột tùy thuộc vào số đối tượng trong frame.
 - `setLayout(new GridLayout(0,2));` // cố định số cột là 2, số dòng mềm dẻo theo số các đối tượng trong frame.

❑ GridBag Layout

- Cách trình bày GridBag Layout cũng trình bày các đối tượng tương tự như Grid Layout: Các đối tượng sẽ được định vị theo vị trí các ô (cell) của một khung lưới (grid). Tuy nhiên, GridBag cho phép ta định kích thước của đối tượng sẽ chiếm bao nhiêu ô và sẽ được đặt ở vị trí nào trong khung lưới.

x=0, y=0	x=1, y=0	x=2, y=0
gridheight=3	gridheight=4	gridheight=3
weighty=0.5	weighty=0.75	weighty=0.5
x=0, y=3 gridheight=3		x=2, y=3 gridheight=3
weighty=0.5	x=1, y=4 gridheight=2	weighty=0.5
	weighty=0.25	



❑ GridBag Layout

Các phương thức cơ bản:

- **GridBagLayout():** Khởi tạo một đối tượng trình bày theo cách gridbag.
- **setConstraints(Component, GridBagConstraints):** Đặt vị trí và kích thước của đối tượng component theo các ràng buộc trong **GridbagConstraints**.



❑ GridBagConstraints

Các thuộc tính và phương thức cơ bản:

- **GridBagConstraints():** Khởi tạo một đối tượng ràng buộc của **GridBag**.
- **gridx/gridy:** Vị trí của cell mà ta muốn đặt đối tượng vào (theo chiều X và chiều Y).
- **gridwidth/gridheight:** Kích thước (vùng trình bày) của đối tượng (Theo chiều rộng và chiều cao).



❑ GridBagConstraints

Các thuộc tính và phương thức cơ bản:

- **fill:** Xác định cách đặt đối tượng, theo 4 cách:
 - **GridBagConstraints.NONE:** Đối tượng không thay đổi kích thước theo các cell nó chiếm.
 - **GridBagConstraints.VERTICAL:** Đối tượng có chiều cao kín vùng nó chiếm
 - **GridBagConstraints.HORIZONTAL:** Đối tượng có chiều rộng kín vùng nó chiếm
 - **GridBagConstraints.BOTH:** Đối tượng có chiều cao và chiều rộng phủ kín vùng nó chiếm.
- **ipadx/ipady:** Định đơn vị tăng giảm kích thước của đối tượng khi khung chứa bị thay đổi kích thước (theo chiều X và chiều Y).
- **insets:** Xác định khoảng cách giữa các cell theo bốn hướng: Trên, dưới, trái, phải.
- **anchor:** Xác định vị trí neo đối tượng khi kích thước khung chứa thay đổi.
 - Bao gồm: **NORTH, NORTHEAST, NORTHWEST, EAST, SOUTH, SOUTHEAST, SOUTHWEST.**
- **weightx/weighty:** Định tỉ trọng tương đối giữa các đối tượng với nhau theo chiều rộng và chiều ngang.



❑ Trình bày Null Layout

Cách trình bày Null Layout sẽ trình bày các đối tượng không theo một quy tắc nào. Tất cả đều do người dùng tự định vị và thiết lập kích thước cho mỗi đối tượng.

- ✓ Định vị đối tượng bằng phương thức setLocation():

`<Đối tượng>.setLocation(Point);`

- ✓ Định kích thước đối tượng bằng phương thức setSize():

`<Đối tượng>.setSize(int, int);`

- ✓ Ngoài ra, có thể vừa định vị, vừa định kích thước cho đối tượng thông qua phương thức:

`<Đối tượng>.setBounds(int, int, int, int);`

- ✓ Trong đó, hai tham số đầu định vị đối tượng, hai tham số sau định kích thước đối tượng.

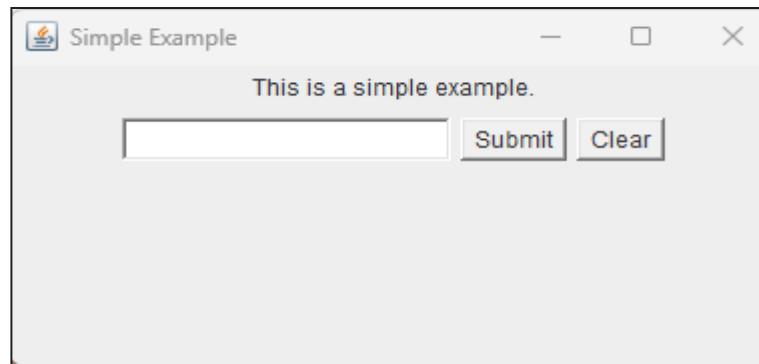


CÁC THÀNH PHẦN CỦA AWT

- ❑ Bài tập thực hành

- ❑ Bài 1:

- ✓ Tạo giao diện đơn giản như hình
- ✓ Thực hành viết chương trình bắt và xử lý sự kiện cho hai nút “Submit” và “Clear” theo các cách khác nhau





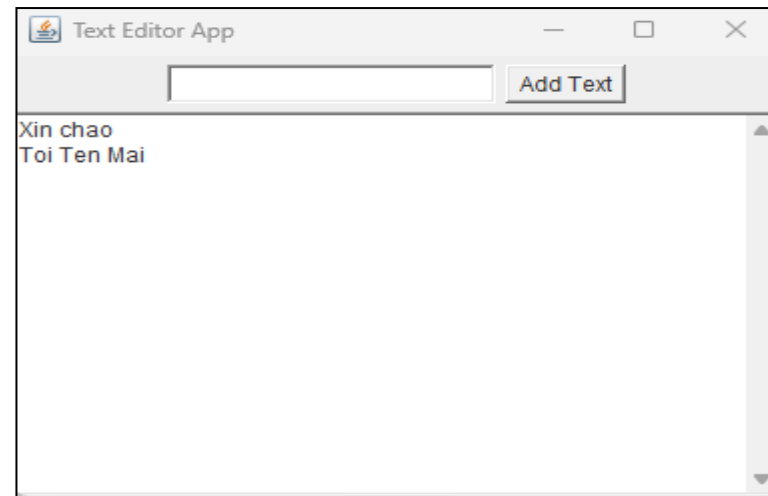
CÁC THÀNH PHẦN CỦA AWT

❑ Bài tập thực hành

Bài 1: Xây dựng ứng dụng có giao diện như sau:

Yêu cầu:

Khi nhấn Button “Add Text”, nội dung trong **TextField** sẽ được thêm vào cuối **TextArea**, cách nhau bởi một dòng mới.





CÁC THÀNH PHẦN CỦA AWT

❑ Bài tập thực hành

Bài 2:

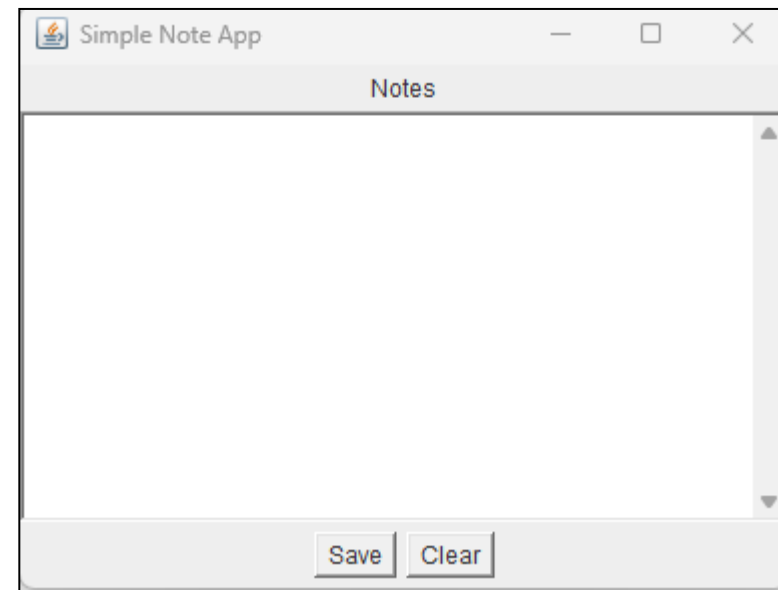
Tạo giao diện cho một ứng dụng ghi chú đơn giản với:

- Một **Label** hiển thị "Notes".
- Một **TextArea** để nhập nội dung ghi chú.
- Hai **Button**: **Save** (lưu ghi chú) và **Clear** (xóa nội dung).

Các thành phần được sắp xếp trong một **Panel**.

Yêu cầu:

Hiển thị giao diện bắt sự kiện cho các nút tương ứng.





TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

Thank you!

THÁI NGUYÊN, THÁNG NĂM